Fashion Retrieval Pipeline

Run this notebook on Google Colab (T4 GPU recommended) to interactively search your fashion index.

```
# 1. Setup and Mount Drive
from google.colab import drive
drive.mount('/content/drive')

!pip install -q "FlagEmbedding>=1.2.11" "faiss-cpu>=1.8.0" "accelerate>=0.34.0"
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force
```

1. System Initialization

Loading the pre-computed FAISS index and the metadata Parquet file into memory. We also initialize the primary text embedding models (BGE-Large) here.

2. Core Search Engine: SigLIP Pipeline

This is the main retrieval function using the **SigLIP** architecture.

1. **Stage 1:** It normalizes the query, embeds it, and uses FAISS to find the top 100 closest text matches.
2. **Stage 2:** It uses **SigLIP** to visually re-rank those 100 images based on actual image-to-text similarity, requiring `padding="max_length"` for its sigmoid logic.

```
!ls -l /content/drive/MyDrive/ | grep -i "fashion"
```

```
drwx------ 4 root root      4096 Jul 16 06:19 Fashion_search_engine
drwx------ 2 root root      4096 Jul 16 08:47 FashionSearchEngine
drwx------ 2 root root      4096 Jul 17 05:54 FashionSearchEngine2
drwx------ 2 root root      4096 Jul 17 08:58 FashionSearchEngine3
```

```
import os
import torch
import numpy as np
import pandas as pd
import faiss
from PIL import Image
from transformers import AutoModelForCausalLM, AutoTokenizer, AutoProcessor, AutoModel
from FlagEmbedding import BGEM3FlagModel
from IPython.display import display, HTML

# Update this if you moved your backup folder!
BACKUP_FOLDER = "/content/drive/MyDrive/Fashion_search_engine/"
if os.path.exists(BACKUP_FOLDER):
    os.chdir(BACKUP_FOLDER)
    print(f"Working directory changed to: {os.getcwd()}")
FAISS_INDEX_PATH = os.path.join(BACKUP_FOLDER, "core", "fashion_search_hnsw.faiss")
if not os.path.exists(FAISS_INDEX_PATH):
    FAISS_INDEX_PATH = os.path.join(BACKUP_FOLDER, "fashion_search_hnsw.faiss")
METADATA_PATH = os.path.join(BACKUP_FOLDER, "core", "images_metadata.parquet")
if not os.path.exists(METADATA_PATH):
    METADATA_PATH = os.path.join(BACKUP_FOLDER, "images_metadata.parquet")

print("Loading FAISS index...")
index = faiss.read_index(FAISS_INDEX_PATH)

print("Loading Metadata...")
df_metadata = pd.read_parquet(METADATA_PATH)
df_metadata.set_index("faiss_id", inplace=True)

device = "cuda" if torch.cuda.is_available() else "cpu"
print(f"Loading Models onto {device.upper()} (This may take a minute)...")

# Normalization Model
llm_model_name = "Qwen/Qwen2.5-0.5B-Instruct"
llm_tokenizer = AutoTokenizer.from_pretrained(llm_model_name)
llm_model = AutoModelForCausalLM.from_pretrained(
    llm_model_name,
    torch_dtype=torch.float16 if device == "cuda" else torch.float32
).to(device)
llm_model.eval()
```

```
# Embedding Model
embed_model = BGEM3FlagModel('BAAI/bge-m3', use_fp16=(device=="cuda"))

# Re-ranking Model
siglip_model_name = "google/siglip2-base-patch16-224"
siglip_processor = AutoProcessor.from_pretrained(siglip_model_name)
siglip_model = AutoModel.from_pretrained(siglip_model_name).to(device)
siglip_model.eval()

print("✅ All systems ready.")
```

```
Working directory changed to: /content/drive/MyDrive/Fashion_search_engine
Loading FAISS index...
Loading Metadata...
Loading Models onto CUDA (This may take a minute)...
[transformers] `torch_dtype` is deprecated! Use `dtype` instead!
Loading weights: 100%                                   290/290 [00:00<00:00, 1192.82it/s]
Download complete: :         0.00B
Reconstruction complete:         0.00B /   0.00B
Fetching 30 files: 100%                                 30/30 [00:00<00:00, 1877.54it/s]
Loading weights: 100%                                   391/391 [00:00<00:00, 2228.67it/s]
[transformers] Model config: bos_token_id must be `None` or an integer within the vocabulary (between 0 and 31999
[transformers] Model config: eos_token_id must be `None` or an integer within the vocabulary (between 0 and 31999
Loading weights: 100%                                   408/408 [00:00<00:00, 4258.81it/s]
✅ All systems ready.
```

```
def normalize_text(raw_query):
    system_prompt = """
You are a semantic query canonicalization model for a fashion search engine.
Convert natural language user queries into clean, pipeline-separated keywords for vector retrieval.

Rules:
1. Extract all clothing items, colors, accessories, and footwear.
2. Keep colors and attributes attached to their garments.
3. PRESERVE styles, occasions, and vibes.
4. PRESERVE locations and environments.
5. Remove conversational filler.
6. Remove actions that are irrelevant to fashion.
7. Output ONLY the canonical keywords separated by " | ".
"""
    base_examples = [
        {"role": "system", "content": system_prompt},
        {"role": "user", "content": "Input: A person in a bright yellow raincoat.\nOutput:"},
        {"role": "assistant", "content": "bright yellow raincoat"},
        {"role": "user", "content": "Input: Professional business attire inside a modern office.\nOutput:"},
        {"role": "assistant", "content": "professional business attire | modern office"},
        {"role": "user", "content": "Input: Someone wearing a blue shirt sitting on a park bench.\nOutput:"},
        {"role": "assistant", "content": "blue shirt | park bench"},
        {"role": "user", "content": f"Input: {raw_query}\nOutput:"}
    ]
    text = llm_tokenizer.apply_chat_template(base_examples, tokenize=False, add_generation_prompt=True)
    inputs = llm_tokenizer([text], return_tensors="pt").to(device)

    with torch.no_grad():
        outputs = llm_model.generate(**inputs, max_new_tokens=64, do_sample=False)

    decoded = llm_tokenizer.decode(outputs[0][inputs.input_ids.shape[-1]:], skip_special_tokens=True).strip()
    cleaned = decoded.strip('"').strip("'")
    if cleaned.lower().startswith("output:"):
        cleaned = cleaned[7:].strip()
    return cleaned


def search_fashion(query, top_k=5, candidate_pool_size=100):
    print(f"\n🔍 Processing Query: '{query}'")
    canonical_query = normalize_text(query)
    print(f"🔹 Canonical: '{canonical_query}'")

    embed_output = embed_model.encode([canonical_query], max_length=8192)
    query_vector = np.array([embed_output['dense_vecs'][0]], dtype=np.float32)
    # L2 Normalize the query for cosine similarity
    query_norm = np.linalg.norm(query_vector)
    if query_norm > 0:
        query_vector = query_vector / query_norm

    distances, indices = index.search(query_vector, candidate_pool_size)
    candidate_faiss_ids = indices[0]
```

```
    valid_ids = [fid for fid in candidate_faiss_ids if fid != -1 and fid in df_metadata.index]
    if not valid_ids:
        print("❌ No matches found.")
        return []

    candidates_df = df_metadata.loc[valid_ids].copy()
    candidates_df['faiss_id'] = candidates_df.index

    print(f"🚀 Re-ranking {len(candidates_df)} candidates using SigLIP...")
    candidate_images = []
    valid_records = []

    for _, row in candidates_df.iterrows():
        img_path = row['image_path']

        # FALLBACK: If dataset was unzipped directly into Colab's /content
        if not os.path.exists(img_path):
            base_name = os.path.basename(img_path)
            possible_path = os.path.join("/content/drive/MyDrive/Fashion_search_engine/data/val_test2020/test", b
            if os.path.exists(possible_path):
                img_path = possible_path

        try:
            img = Image.open(img_path).convert("RGB")
            candidate_images.append(img)

            # ---> WE ADDED THE CANONICAL CAPTION HERE <---
            valid_records.append({
                "faiss_id": row['faiss_id'],
                "image_path": img_path,
                "caption": row['original_caption'],
                "canonical": row['canonical_caption']
            })
        except Exception:
            continue

    if not candidate_images:
        print("❌ No valid images found. Check your dataset paths!")
        return []

    inputs = siglip_processor(
        text=[query],
        images=candidate_images,
        padding="max_length",
        return_tensors="pt"
    ).to(device)

    with torch.no_grad():
        outputs = siglip_model(**inputs)
        logits = outputs.logits_per_image.squeeze(-1)
        scores = torch.sigmoid(logits).cpu().numpy()

    for i, rec in enumerate(valid_records):
        rec['rerank_score'] = float(scores[i])

    ranked_results = sorted(valid_records, key=lambda x: x['rerank_score'], reverse=True)
    return ranked_results[:top_k]
```

```
query = "a person wearing a bright yellow raincoat"
canonical_query = normalize_text(query)
embed_output = embed_model.encode([canonical_query], max_length=8192)
query_vector = np.array([embed_output["dense_vecs"][0]], dtype=np.float32)
query_vector = query_vector / np.linalg.norm(query_vector)

distances, indices = index.search(query_vector, 100)
valid_ids = [fid for fid in indices[0] if fid != -1 and fid in df_metadata.index]

print("Top 20 FAISS candidates (before SigLIP):")
for rank, fid in enumerate(valid_ids[:20], start=1):
    row = df_metadata.loc[fid]
    print(f"{rank:2d} | sim={distances[0][rank-1]:.4f} | {row['canonical_caption']}")
    print(f"     {row['original_caption'][:120]}...")
```

```
Top 20 FAISS candidates (before SigLIP):
 1 | sim=0.7332 | yellow raincoat | black trousers | yellow bucket hat | white sneakers
     A person is standing wearing a yellow raincoat, black trousers, a yellow bucket hat, and white sneakers....
 2 | sim=0.6865 | yellow jacket | light blue jeans | bright spotlight | dark background
     A model walks on a runway wearing a yellow jacket and light blue jeans, with a bright spotlight illuminating
 3 | sim=0.6772 | bright yellow strapless dress | sandy beach | water | background
     A woman in a bright yellow strapless dress stands on a sandy beach, holding the fabric of her dress as the w
 4 | sim=0.6753 | bright yellow hooded jacket | black trousers | white shoes
```

```
    A man is standing in a plain white background wearing a bright yellow hooded jacket, black trousers, and whi
 5 | sim=0.6743 | bright yellow gown | lace detail | short ruffled sleeves
    A model stands on a runway wearing a bright yellow, floor-length gown with lace detailing and short, ruffled
 6 | sim=0.6593 | bright yellow jacket | black pants | purple stage lighting | runway | black bag
    A woman in a bright yellow jacket and black pants is walking on a runway, carrying a black bag, under purple
 7 | sim=0.6569 | light blue denim jacket | black sweater | dark pants | colorful plaid scarf | chain-link fence
    A person wearing a light blue denim jacket, a black sweater, dark pants, and a colorful plaid scarf, stands
 8 | sim=0.6526 | yellow dress | denim jacket | yellow shirt | yellow shirt in background
    A woman is standing in an indoor setting, wearing a yellow dress with a denim jacket, and a person in a yell
 9 | sim=0.6494 | bright yellow hooded coat | cream-colored pleated skirt | light-colored ankle boots | plain whi
    A model stands facing forward wearing a bright yellow hooded coat, a cream-colored pleated skirt, and light-
10 | sim=0.6477 | bright yellow dress | matching yellow shorts | dark stage | white cello
    A woman in a bright yellow sleeveless dress and matching yellow shorts stands on a dark stage, holding a whi
11 | sim=0.6475 | yellow blazer | ripped jeans | colorful scarf | sunglasses | white handbag | gold watch | brigh
    A woman stands in a brightly lit indoor space wearing a yellow blazer, ripped jeans, a colorful scarf, and s
12 | sim=0.6463 | yellow long coat | red tights | red hat | tan boots | green corrugated wall
    A woman stands in front of a green corrugated wall, wearing a yellow long coat, red tights, and a red hat, w
13 | sim=0.6416 | bright yellow coat | wide-leg pants | black backpack | water bottle | city street | other peopl
    A woman in a bright yellow coat and matching wide-leg pants walks down a city street, with a black backpack
14 | sim=0.6352 | colorful floral jacket | bright blue trousers | dark blue shoes
    A model walks down a runway wearing a colorful floral jacket, bright blue trousers, and dark blue shoes....
15 | sim=0.6338 | yellow coat | white pants | yellow scarf | runway | crowd of spectators seated | background
    A model walks down a runway in a yellow coat, white pants, and a yellow scarf, with a crowd of spectators se
16 | sim=0.6326 | white cable-knit sweater | grey trousers | sunglasses | dirt path | field of yellow wildflowers
    A woman wearing a white cable-knit sweater, grey trousers, and sunglasses is walking along a dirt path throu
17 | sim=0.6308 | bright yellow form-fitting halter-neck gown | gold belt | chunky gold necklace | short styled h
    A woman stands with her hands on her hips, wearing a bright yellow, form-fitting, halter-neck gown with a go
18 | sim=0.6295 | leopard-print jacket | colorful tie-dye leggings | yellow flower | dimly lit tunnel
    A woman stands in a dimly lit, arched tunnel wearing a leopard-print jacket, colorful tie-dye leggings, and
19 | sim=0.6282 | light yellow fur coat | matching fur hat | red star emblem | silver bracelet | textured off-whi
    A woman stands in front of a textured, off-white wall, wearing a light yellow fur coat, a matching fur hat w
20 | sim=0.6259 | red jacket | blue jeans | bright blue sneakers | blue scarf | park
    A woman stands in a park wearing a red jacket, blue jeans, and bright blue sneakers, with a blue scarf aroun
```

EXAMPLES TO TEST THE RETRIVAL

- Attribute Specific: "A person in a bright yellow raincoat."
- Contextual/Place: "Professional business attire inside a modern office."
- Complex Semantic: "Someone wearing a blue shirt sitting on a park bench."
- Style Inference: "Casual weekend outfit for a city walk."
- Compositional: "A red tie and a white shirt in a formal setting."

```
print("FAISS vectors:", index.ntotal)
print("Metadata rows:", len(df_metadata))

mask = df_metadata["original_caption"].str.contains(r"yellow|raincoat", case=False, na=False)
print("Rows matching yellow|raincoat:", mask.sum())
display(df_metadata.loc[mask, ["original_caption", "canonical_caption"]].head(20))
```

```
FAISS vectors: 3209
Metadata rows: 3209
Rows matching yellow|raincoat: 215
```

faiss_id	original_caption	canonical_caption
5	A woman stands against a brown, vertically sla...	flowering dress | black jacket | black legging...
16	A model walks down a runway in a yellow coat, ...	yellow coat | white pants | yellow scarf | run...
18	A woman in a white fluffy jacket, yellow top, ...	white fluffy jacket | yellow top | black rippe...
19	A woman stands on a cobblestone city street we...	red green black plaid hooded cape | black polk...
41	A model walks on a runway wearing a patterned ...	patterned green and white jacket | white wide-...
69	A woman stands in front of a white vertical wo...	yellow sleeveless dress | brown belt | pink sa...
103	A woman with short black hair is walking away ...	black leather jacket | yellow top | long grey ...
105	A model walks down a runway wearing a colorful...	colorful patterned top | bright yellow pants |...
174	A woman in a white v-neck top and black pants ...	white v-neck top | black pants | city street |...
176	A woman is walking on a runway wearing a brigh...	bright pink long-sleeved top | yellow cuffs | ...
225	A woman stands on a city street wearing a flor...	flowering dress | brown cardigan | brown belt ...
226	A model walks down a runway wearing a beige tr...	beige trench coat | yellow collared shirt | be...
235	A model walks down a runway wearing a dark bro...	dark brown long-sleeved top | white pleated sk...
236	A model walks down a runway wearing a black lo...	black long-sleeved dress | white collar | yell...
454	A woman in a yellow dress and black boots sits...	yellow dress | black boots | stone step | graf...
275	A model stands on a runway wearing a bright ye...	bright yellow gown | lace detail | short ruffl...
281	A model walks down a runway wearing a straples...	strapless yellow and blue dress | black belt |...
282	A model walks down a runway wearing a yellow p...	yellow poncho | black leggings | black strappy...
283	A woman is leaning against a pale yellow wall,...	red sleeveless dress | V-neck | twisted front ...
308	A man is walking down a runway wearing a yello...	yellow t-shirt | black and white checkered pat...

```
# Example 2
query = "a person wearing a bright yellow raincoat"
results = search_fashion(query, top_k=5)

for i, res in enumerate(results):
    print(f"\n{'='*70}\nRank {i+1} | Score: {res['rerank_score']:.4f}")
    print(f"Rich Caption: {res['caption']}")
    print(f"Normalized:   {res['canonical']}")

    try:
        img = Image.open(res['image_path'])
        img.thumbnail((300, 300))
        display(img)
    except:
        print("[Image not found locally]")
```

```
🔍 Processing Query: 'a person wearing a bright yellow raincoat'
🔹 Canonical: 'bright yellow raincoat'
🚀 Re-ranking 100 candidates using SigLIP...

======================================================================
Rank 1 | Score: 0.7906
Rich Caption: A model stands facing forward wearing a bright yellow hooded coat, a cream-colored pleated skirt,
Normalized:   bright yellow hooded coat | cream-colored pleated skirt | light-colored ankle boots | plain white
```

```
# Example 1
query = "Professional business attire inside a modern office"
results = search_fashion(query, top_k=5)

for i, res in enumerate(results):
    print(f"\n{'='*70}\nRank {i+1} | Score: {res['rerank_score']:.4f}")
    print(f"Rich Caption: {res['caption']}")
    print(f"Normalized:   {res['canonical']}")

    try:
        img = Image.open(res['image_path'])
        img.thumbnail((300, 300))
        display(img)
    except:
        print("[Image not found locally]")
```

```
======================================================================
Rank 3 | Score: 0.3939
Rich Caption: A person is standing wearing a yellow raincoat, black trousers, a yellow bucket hat, and white sne
Normalized:   yellow raincoat | black trousers | yellow bucket hat | white sneakers
```

```
======================================================================
Rank 4 | Score: 0.0720
Rich Caption: A woman stands in front of a green corrugated wall, wearing a yellow long coat, red tights, and a
Normalized:   yellow long coat | red tights | red hat | tan boots | green corrugated wall
```

```
======================================================================
Rank 5 | Score: 0.0255
Rich Caption: A model walks down a runway in a yellow coat, white pants, and a yellow scarf, with a crowd of spe
Normalized:   yellow coat | white pants | yellow scarf | runway | crowd of spectators seated | background
```

```
🔍 Processing Query: 'Professional business attire inside a modern office'
🔹 Canonical: 'professional business attire | modern office'
🚀 Re-ranking 100 candidates using SigLIP...

======================================================================
Rank 1 | Score: 0.6933
Rich Caption: A man in a navy blazer, grey trousers, and brown shoes stands in a modern office with a tablet in
Normalized:   navy blazer | grey trousers | brown shoes | modern office | tablet
```

```
# Example 3
query = "Someone wearing a blue shirt sitting on a park bench"
results = search_fashion(query, top_k=5)

for i, res in enumerate(results):
    print(f"\n{'='*40}\nRank {i+1} | Score: {res['rerank_score']:.4f}")
    print(f"Caption: {res['caption']}")
    print(f"Normalized:   {res['canonical']}")

    try:
        img = Image.open(res['image_path'])
        img.thumbnail((300, 300))
        display(img)
    except:
        print("[Image not found locally]")
```

```
======================================================================
Rank 3 | Score: 0.5229
Rich Caption: A man in a dark grey blazer, dark trousers, and brown shoes stands in a modern office with large v
Normalized:   dark grey blazer | dark trousers | brown shoes | modern office | large windows | holding tablet
```

```
======================================================================
Rank 4 | Score: 0.3667
Rich Caption: A man in a grey suit with a blue tie stands in a busy fashion workshop, leaning on a table with la
Normalized:   grey suit | blue tie | busy fashion workshop | large rolls of fabric | other workers
```

```
======================================================================
Rank 5 | Score: 0.1658
```

```
Rich Caption: A woman with short red hair is standing outdoors in a modern building, wearing a black blazer ove
Normalized:   short red hair | modern building | black blazer | black and white houndstooth top | black pants |
```

```
🔍 Processing Query: 'Someone wearing a blue shirt sitting on a park bench'
🔹 Canonical: 'blue shirt | park bench'
🚀 Re-ranking 100 candidates using SigLIP...

========================================
Rank 1 | Score: 0.9582
Caption: A man with a beard is sitting on a park bench wearing a light blue button-down shirt and dark pants, w
```

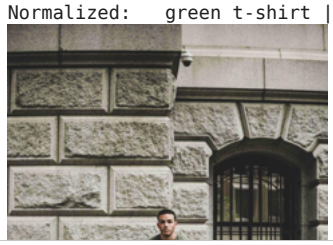
```
# Example 4
query = "Casual weekend outfit for a city walk"
results = search_fashion(query, top_k=5)

for i, res in enumerate(results):
    print(f"\n{'='*70}\nRank {i+1} | Score: {res['rerank_score']:.4f}")
    print(f"Rich Caption: {res['caption']}")
    print(f"Normalized:   {res['canonical']}")

    try:
        img = Image.open(res['image_path'])
        img.thumbnail((300, 300))
        display(img)
    except:
        print("[Image not found locally]")
```

```
========================================
Rank 2 | Score: 0.7486
Caption: A woman is sitting on a white bench in a park, wearing a blue and white sports jersey with the number 8
Normalized:   blue and white sports jersey | number 86 | light blue shorts | tan ankle boots | park
```

```
========================================
Rank 3 | Score: 0.1659
Caption: A woman stands on a white bench wearing a blue and white sports jersey with the number 86, light blue
Normalized:   blue and white sports jersey | number 86 | light blue denim shorts | brown boots | white bench |
```

```
========================================
Rank 4 | Score: 0.1566
Caption: A woman wearing a grey t-shirt, blue knee-high socks, white platform shoes, and a white hat is sitting
Normalized:   grey t-shirt | blue knee-high socks | white platform shoes | white hat | green park bench | legs
```

```
========================================
Rank 5 | Score: 0.0322
Caption: A man is sitting on a stone bench in front of a stone building with a barred window, wearing a green t
Normalized:   green t-shirt | blue jeans | black sneakers | stone building | stone bench | barred window
```



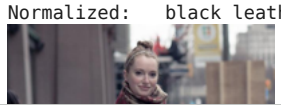
```
🔍 Processing Query: 'Casual weekend outfit for a city walk'
🔹 Canonical: 'casual weekend outfit | city walk'
🚀 Re-ranking 100 candidates using SigLIP...

======================================================================
Rank 1 | Score: 0.5719
Rich Caption: A woman wearing a black leather jacket, a black scarf, black leggings, and black boots is walking
Normalized:   black leather jacket | black scarf | black leggings | black boots | city street
```

```
# Example 5
query = "A red tie and a white shirt in a formal setting."
results = search_fashion(query, top_k=5)

for i, res in enumerate(results):
    print(f"\n{'='*70}\nRank {i+1} | Score: {res['rerank_score']:.4f}")
    print(f"Rich Caption: {res['caption']}")
    print(f"Normalized:   {res['canonical']}")

    try:
        img = Image.open(res['image_path'])
        img.thumbnail((300, 300))
        display(img)
    except:
        print("[Image not found locally]")
```

```
======================================================================
Rank 3 | Score: 0.3676
Rich Caption: A woman in a black puffer jacket, light blue jeans, and black boots is walking on a city street, l
Normalized:   black puffer jacket | light blue jeans | black boots | city street | pink bag | purple scarf
```

```
======================================================================
Rank 4 | Score: 0.3470
Rich Caption: A woman stands on a city street holding a purple clutch, wearing a black striped jacket, black pai
Normalized:   purple clutch | black striped jacket | black pants | silver shoes | city street | dog nearby
```

```
======================================================================
Rank 5 | Score: 0.2155
Rich Caption: A woman is walking on a city street, wearing a black leather jacket, blue jeans, white sneakers, a
Normalized:   black leather jacket | blue jeans | white sneakers | red and brown patterned scarf | brown crossb
```



```
🔍 Processing Query: 'A red tie and a white shirt in a formal setting.'
🔹 Canonical: 'red tie | white shirt | formal setting'
🚀 Re-ranking 100 candidates using SigLIP...

======================================================================
Rank 1 | Score: 0.7932
Rich Caption: A man stands in front of a green car, wearing a white dress shirt, dark trousers, and a red tie.
Normalized:   white dress shirt | dark trousers | red tie | green car
```

```
======================================================================
Rank 2 | Score: 0.0195
Rich Caption: A man in a grey suit with a white shirt, a dark tie, and a striped tie, standing in a backstage a
Normalized:   grey suit | white shirt | dark tie | striped tie | backstage area | professional lighting equipmer
```

Double-click (or enter) to edit

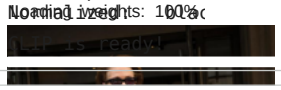
3. Core Search Engine: CLIP Pipeline

Here, we swap out the re-ranking engine to use **OpenAI's CLIP (clip-vit-large-patch14)**.

```
from transformers import CLIPProcessor, CLIPModel
import torch

device = "cuda" if torch.cuda.is_available() else "cpu"

print("Loading OpenAI CLIP...")
clip_model_name = "openai/clip-vit-large-patch14"
clip_processor = CLIPProcessor.from_pretrained(clip_model_name)
clip_model = CLIPModel.from_pretrained(clip_model_name).to(device)
clip_model.eval()
print("CLIP is ready!")
```

```
Loading OpenAI CLIP...
Loading weights: 100%|██████████| 590/590 [00:00<00:00, 4383.65it/s]
CLIP is ready!
```

```
Rank 3 | Score: 0.0094
Rich Caption: A woman in a black pinstripe blazer, white shirt, black trousers, and red accessories walks out o
Normalized:   black pinstripe blazer | white shirt | black trousers | red accessories | building
```



```
def normalize_text(raw_query):
    system_prompt = """
You are a semantic query canonicalization model for a fashion search engine.
Convert natural language user queries into clean, pipeline-separated keywords for vector retrieval.

Rules:
1. Extract all clothing items, colors, accessories, and footwear.
2. Keep colors and attributes attached to their garments.
3. PRESERVE styles, occasions, and vibes.
4. PRESERVE locations and environments.
5. Remove conversational filler.
6. Remove actions that are irrelevant to fashion.
7. Output ONLY the canonical keywords separated by " | ".
"""
    base_examples = [
        {"role": "system", "content": system_prompt},
        {"role": "user", "content": "Input: A person in a bright yellow raincoat.\nOutput:"},
        {"role": "assistant", "content": "bright yellow raincoat"},
        {"role": "user", "content": "Input: Professional business attire inside a modern office.\nOutput:"},
        {"role": "assistant", "content": "professional business attire | modern office"},
        {"role": "user", "content": "Input: Someone wearing a blue shirt sitting on a park bench.\nOutput:"},
        {"role": "assistant", "content": "blue shirt | park bench"},
        {"role": "user", "content": f"Input: {raw_query}\nOutput:"}
    ]
    text = llm_tokenizer.apply_chat_template(base_examples, tokenize=False, add_generation_prompt=True)
    inputs = llm_tokenizer([text], return_tensors="pt").to(device)

    with torch.no_grad():
        outputs = llm_model.generate(**inputs, max_new_tokens=64, do_sample=False)
```

```
    decoded = llm_tokenizer.decode(outputs[0][inputs.input_ids.shape[-1]:], skip_special_tokens=True).strip()
    cleaned = decoded.strip('"').strip("'")
    if cleaned.lower().startswith("output:"):
        cleaned = cleaned[7:].strip()
    return cleaned

def search_fashion(query, top_k=5, candidate_pool_size=100):
    print(f"\n🔍 Processing Query: '{query}'")
    canonical_query = normalize_text(query)
    print(f"🔹 Canonical: '{canonical_query}'")

    embed_output = embed_model.encode([canonical_query], max_length=8192)
    query_vector = np.array([embed_output['dense_vecs'][0]], dtype=np.float32)
    # L2 Normalize the query for cosine similarity
    query_norm = np.linalg.norm(query_vector)
    if query_norm > 0:
        query_vector = query_vector / query_norm

    distances, indices = index.search(query_vector, candidate_pool_size)
    candidate_faiss_ids = indices[0]

    valid_ids = [fid for fid in candidate_faiss_ids if fid != -1 and fid in df_metadata.index]
    if not valid_ids:
        print("❌ No matches found.")
        return []

    candidates_df = df_metadata.loc[valid_ids].copy()
    candidates_df['faiss_id'] = candidates_df.index

    print(f"🚀 Re-ranking {len(candidates_df)} candidates using CLIP...")
    candidate_images = []
    valid_records = []

    for _, row in candidates_df.iterrows():
        img_path = row['image_path']

        # FALLBACK: If dataset was unzipped directly into Colab's /content
        # (e.g., /content/data/val_test2020/test/)
        if not os.path.exists(img_path):
            base_name = os.path.basename(img_path)
            # Adjust this path depending on where your dataset is stored on Colab/Drive!
            possible_path = os.path.join("/content/drive/MyDrive/Fashion_search_engine/data/val_test2020/test",
            if os.path.exists(possible_path):
                img_path = possible_path

        try:
            img = Image.open(img_path).convert("RGB")
            candidate_images.append(img)
            valid_records.append({"faiss_id": row['faiss_id'], "image_path": img_path, "caption": row['original_
        except Exception:
            continue

    if not candidate_images:
        print("❌ No valid images found. Check your dataset paths!")
        return []

    # --- CLIP INFERENCE ---
    inputs = clip_processor(
        text=[query],
        images=candidate_images,
        padding=True, # CLIP uses standard padding!
        return_tensors="pt"
    ).to(device)

    with torch.no_grad():
        outputs = clip_model(**inputs)
        # CLIP's raw logits are the scaled cosine similarities
        logits = outputs.logits_per_image.squeeze(-1)
        scores = logits.cpu().numpy()

    for i, rec in enumerate(valid_records):
        rec['rerank_score'] = float(scores[i])

    ranked_results = sorted(valid_records, key=lambda x: x['rerank_score'], reverse=True)

    # Optional: Print out the top results for an easy check
    print("\n" + "-"*40)
    print("🏆 Top CLIP Results:")
    for i, res in enumerate(ranked_results[:top_k]):
        print(f"[{i+1}] Score: {res['rerank_score']:.4f} | Path: {res['image_path'].split('/')[-1]}")
        print(f"    Caption: {res['caption'][:120]}...")
    print("-" * 40 + "\n")
```

```
    return ranked_results[:top_k]
```

```
import matplotlib.pyplot as plt
from PIL import Image

def test_and_visualize(query):
    # Run the search
    results = search_fashion(query, top_k=5)

    if not results:
        print("No results to display.")
        return

    # Set up a wide plot for the images
    fig, axes = plt.subplots(1, len(results), figsize=(20, 5))

    # Handle the case if there is only 1 result
    if len(results) == 1:
        axes = [axes]

    fig.suptitle(f"Top CLIP Results for: '{query}'", fontsize=16, fontweight='bold')

    for ax, res in zip(axes, results):
        # Open and display the image
        img = Image.open(res['image_path']).convert("RGB")
        ax.imshow(img)
        ax.axis("off")

        # Format a short caption so it fits nicely above the image
        score = res['rerank_score']
        short_caption = res['caption'][:60] + "..." if len(res['caption']) > 60 else res['caption']

        ax.set_title(f"Score: {score:.2f}\n{short_caption}", fontsize=10, wrap=True)

    plt.tight_layout()
    plt.show()

# --- RUN YOUR TESTS HERE ---

# Test 1: Your original query
test_and_visualize("a person wearing a bright yellow raincoat")

# Test 2: Try something different to test CLIP's zero-shot power!
# test_and_visualize("a woman in a red dress on the beach")
# test_and_visualize("professional business attire in a modern office")
```